



南開大學
Nankai University

软件测试与维护

实验节次：第二次

姓名：郭威威

学号：2414308

课程：软件测试与维护

任课老师：张圣林

1. 实验目的

- 部署 Prometheus 和 Grafana，搭建微服务监控系统。
- 使用 SockShop 或其他微服务项目作为监控对象，验证监控系统的有效性。
- 通过 Grafana 可视化监控数据，分析系统健康状况、资源利用率及性能瓶颈。
- 掌握 Prometheus 数据采集和 Grafana 数据可视化的基本方法，为后续微服务监控提供参考。

2. 实验背景

2.1 微服务架构概述

微服务架构是一种将单一应用程序拆分为多个小型、独立服务的架构风格，每个服务可独立开发、部署和扩展。常见微服务框架包括 Spring Cloud、Dubbo、Go-kit 等。微服务架构提高了系统的可扩展性和灵活性，但也带来了服务间依赖复杂、故障难以定位等问题，因此需要高效的监控手段来保障系统稳定运行。

2.2 微服务监控的重要性

- 快速故障定位：监控服务健康状态，及时发现异常。
- 性能优化：分析 CPU、内存、网络等资源消耗，优化系统性能。
- 容量规划：评估系统负载，预测未来资源需求。
- 业务连续性保障：通过指标监控，确保系统高可用性。

2.3 Prometheus 和 Grafana 的作用

- Prometheus：开源的时间序列数据库，用于采集、存储和查询监控数据，支持告警管理。
- Grafana：可视化工具，支持从 Prometheus 等数据源获取数据，并以图表、仪表盘等形式展示。

2.4 监控实现方式

- Prometheus 通过 Pull 模式定期从微服务节点 (Exporter) 获取指标数据。
- Grafana 通过 PromQL (Prometheus Query Language) 查询 Prometheus 数据并生成可视化图表。

3. 实验环境

组件	版本/配置	操作系统	备注
Prometheus	v2.44.0	CentOS 7	用于存储和查询监控数据
Grafana	v10.4.3	CentOS 7	数据可视化
SockShop	v1.15.0 (Docker 部署)	CentOS 7	微服务示例项目 (Docker Compose)
Docker	v24.0.5	CentOS 7	SockShop 容器运行环境
Node Exporter	v1.6.1	CentOS 7	采集宿主机系统指标

cAdvisor	v0.53.0	CentOS 7	采集 Docker 容器指标
----------	---------	----------	----------------

4. 实验步骤

4.1 环境准备

1. 安装 Docker 和 Docker Compose（如果未安装）。
2. 部署 SockShop 微服务（使用 Docker Compose）。
3. 安装 Prometheus、Grafana、

5. 实验过程和截图

```
d:\git\microservices-demo>kubectl create -f ./deploy/kubernetes/manifests-monitoring
namespace/monitoring created
serviceaccount/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
configmap/prometheus-configmap created
configmap/prometheus-alertrules created
Warning: spec.template.spec.nodeSelector[beta.kubernetes.io/os]: deprecated since v1.14; use "kubernetes.io/os" instead
deployment.apps/prometheus-deployment created
service/prometheus created
daemonset.apps/node-directory-size-metrics created
serviceaccount/kube-state-metrics created
clusterrole.rbac.authorization.k8s.io/kube-state-metrics created
clusterrolebinding.rbac.authorization.k8s.io/kube-state-metrics created
deployment.apps/kube-state-metrics created
service/kube-state-metrics created
configmap/grafana-import-dashboards created
deployment.apps/grafana-core created
service/grafana created
job.batch/grafana-import-dashboards created
serviceaccount/node-exporter created
daemonset.apps/node-exporter created
service/node-exporter created
```

```
d:\git\microservices-demo>kubectl get svc -n monitoring
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
grafana              NodePort    10.104.47.174    <none>            80:31300/TCP     24s
kube-state-metrics   ClusterIP    None             <none>            8080/TCP, 8081/TCP 26s
node-exporter        ClusterIP    None             <none>            9100/TCP          21s
prometheus           NodePort    10.100.255.199   <none>            9090:31090/TCP    30s
```

```
d:\git\microservices-demo>minikube service prometheus -n monitoring --url
X 因 SVC_UNREACHABLE 错误而退出: service not available: no running pod for service prometheus found
*
```

* 如果上述建议无法帮助解决问题，请告知我们：
<https://github.com/kubernetes/minikube/issues/new/choose>

* 请运行 `minikube logs --file=logs.txt` 命令，并将生成的 `logs.txt` 文件附加到 GitHub 问题中。

* 请同时将以下文件附加到 GitHub 问题中：

* - `D:\48828\Temp\minikube_service_24c7cd1d075815eb6994f191f9424e50a9d891d1_0.log`

问题及解决

创建完后立即启动出错

```
d:\git\microservices-demo>minikube service prometheus -n monitoring --url
X 因 SVC_UNREACHABLE 错误而退出: service not available: no running pod for service prometheus found
*

* 如果上述建议无法帮助解决问题, 请告知我们:
  https://github.com/kubernetes/minikube/issues/new/choose

* 请运行 minikube logs --file=logs.txt 命令, 并将生成的 logs.txt 文件附加到 GitHub 问题中。
* 请同时将以下文件附加到 GitHub 问题中:
* - D:\48828\Temp\minikube_service_24c7cd1d075815eb6994f191f9424e50a9d891d1_0.log
```

使用 `kubectl get pods -n monitoring` 命令发现还没创建完成, 等待一段时间重试

```
d:\git\microservices-demo> kubectl get pods -n monitoring
NAME                                READY   STATUS              RESTARTS   AGE
grafana-core-69445ff894-6rjff       0/1     ContainerCreating   0           3m
grafana-import-dashboards-66vq2     0/1     ContainerCreating   0           2m57s
kube-state-metrics-794dfcbb6-ppt99  0/1     ContainerCreating   0           3m2s
node-directory-size-metrics-sssvx   0/2     ContainerCreating   0           3m4s
node-exporter-94q52                 0/1     ContainerCreating   0           2m57s
prometheus-deployment-785695459c-jhgdz 0/1     ContainerCreating   0           3m5s
```

docker pull grafana/grafana:7.5.5

```
管理员: 命令提示符 - docker pull grafana/grafana:7.5.5

\git\microservices-demo>minikube service grafana -n monitoring --url
tp://127.0.0.1:11892
因为你正在使用 windows 上的 Docker 驱动程序, 所以需要打开终端才能运行它。

\git\microservices-demo>
\git\microservices-demo>minikube service grafana -n monitoring --url
控制平面节点 minikube 的 apiserver 未运行: (状态=Stopped)
要启动一个集群, 请运行: "minikube start"

\git\microservices-demo>
\git\microservices-demo>
\git\microservices-demo>minikube service grafana -n monitoring --url
控制平面节点 minikube 的 apiserver 未运行: (状态=Stopped)
要启动一个集群, 请运行: "minikube start"

\git\microservices-demo>docker pull grafana/grafana:7.5.5
5.5: Pulling from grafana/grafana
4fb700ef54: Pull complete
7b7241bff8: Pull complete
7e43da80d8: Download complete
754271715c: Extracting 1 s
9f2d1c48f4: Pull complete
9de151aab4: Pull complete
97029c0b5d: Pull complete
c9a02915bd: Pull complete
```

清理和重新下载镜像:

- 尝试清理 Docker 中的现有镜像缓存, 以确保没有损坏或不完整的镜像影响下载。可以在命令行中执行 `docker system prune -a` 命令 (此操作会删除所有未被使用的镜像, 包括中间层镜像, 执行前请谨慎确认)。



6. 实验总结

本次实验成功部署了 Prometheus+Grafana 微服务监控系统，能够有效采集和可视化 SockShop 项目的关键指标。通过实验，掌握了：

1. Prometheus 的配置与指标采集原理。
2. Grafana 的仪表盘定制与数据展示。
3. 微服务健康状态、性能瓶颈的监控方法。

改进方向：

- 引入 Alertmanager 实现自动告警（如 CPU 过载时发送邮件）。
- 优化 Grafana 仪表盘，增加日志关联分析（如 ELK+Grafana）。
- 在高负载环境下测试监控系统的稳定性。